# Discovery Executive Summary

**Project:** dicovery-123 · **Generated:** 24/07/2026, 18:35:34

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 2 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

## Portfolio Overview

| # | Analysis | Overall Rating | Hotspot Score |
|---|---|---|---|
| 1 | Architecture & Design Analysis | HIGH RISK | — |
| 2 | Code Quality & Complexity Analysis | HIGH RISK | 42 / 100 — Moderate |

---

## 1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

### High Risk

Driven by High-Risk Missing Service/Repository layers (H2/H3), Domain Boundary Violations (H8), Shared DB Coupling (H9), and Dual-API duplication (H10).

EXECUTIVE SUMMARY

Klearcom is a modular monolith (Discovery IVR + Connect TFN) with a thin Laravel 12 API, a React 19 SPA, and a parallel Express `dev-api` that re-implements the same workflows. Controllers are short by LOC but fat by responsibility: Eloquent and KPI math live in HTTP handlers, Modules folders are documentation stubs only, and there are zero repository classes. The dominant risk is change amplification from duplicated business logic across Laravel, Express, and the SPA, plus shared Mongo collections and cross-domain dashboard/legacy reports that erase bounded-context ownership. Frontend has a usable `api/client` and React Query, but legacy class/widget patterns and hard-coded endpoint strings in pages still couple UI to backend shapes.

## 1.1 Benchmark Ratings Summary

| # | Hotspot | Primary KPI | GOOD | MODERATE | HIGH RISK | Measured | Rating |
|---|---|---|---|---|---|---|---|
| H1 | Fat Controllers | Avg LOC per controller | <150 | 150–300 | >300 | 62 LOC (Laravel avg); Express `server.js` 224 LOC | MODE |
| H2 | Missing Service Layer | Controllers accessing repos/models | <10 | 10–20 | >20 | 25 handler methods with direct model/store access | HIGH |
| H3 | Missing Repository Pattern | Direct DB access points | <10 | 10–20 | >20 | 35+ Eloquent/store/Mongo access points; 0 repositories | HIGH |
| H4 | Circular Dependencies | Dependency cycles | 0 | 1–3 | >3 | 0 | GOOD |
| H5 | Shared Utility Abuse | Utility files w/ business logic | 0 | 1–5 | >5 | 2 ( `LegacyDataMapper` , shared `buildTree` helpers) | MODE |
| H6 | Direct SQL in Controllers | ORM/repository compliance % | >90% | 60–90% | <60% | ~38% of queries outside controllers | HIGH |
| H7 | God Classes | Classes >1000 LOC | 0 | 1–3 | >3 | 0 (largest `server.js` 224 LOC) | GOOD |
| H8 | Domain Boundary Violations | Cross-domain access points | 0 | 1–5 | >5 | 8 cross-domain access points | HIGH |
| H9 | Shared Database Coupling | Tables shared across domains | <10% | 10–30% | >30% | 37.5% (3/8 stores shared via Mongo `module` ) | HIGH |
| H10 | Dual API Duplication (additional) | Duplicated workflows across Laravel ↔ Express | 0 | 1–3 | >3 | 5+ workflows duplicated | HIGH |
| F1 | Business Logic in Components | Avg LOC per component | <150 | 150–300 | >300 | 74 LOC avg | GOOD |
| F2 | Missing Frontend Service/Data Layer | Components w/ inline API calls | <10 | 10–20 | >20 | 6 components with hard-coded paths | GOOD |
| F3 | God / Oversized Components | Components >400 LOC | 0 | 1–3 | >3 | 0 (largest ConnectPage 208 LOC) | GOOD |
| F4 | Prop Drilling / Global State Abuse | Max prop-drilling depth | ≤2 | 3–4 | >4 | 2 levels; small Zustand UI store | GOOD |
| F5 | Legacy / Inconsistent | Legacy-pattern components | 0 | 1–10 | >10 | 2 ( `LegacyMonitorPoller` , `LegacyDashboardWidget` ) | MODE |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| | Component Patterns | | | | | | |

## 1.4 Actions Required

| Hotspot | Action | Rating | Priority |
|---|---|---|---|
| H1 Fat Controllers | Extract reachability/tree/KPI logic from `ConnectController`, `LegacyReportController`, and Express `server.js` into Application Services | MODERATE | HIGH |
| H2 Missing Service Layer | Create `DiscoveryJobService`, `ConnectMonitorService`, `DashboardKpiService`; stop Eloquent/store use in handlers | HIGH RISK | CRITICAL |
| H3 Missing Repository Pattern | Add repository interfaces for Eloquent models; bind in `AppServiceProvider`; keep Mongo behind repository APIs | HIGH RISK | CRITICAL |
| H5 Shared Utility Abuse | Replace `LegacyDataMapper` extract() with DTOs; single `IvrTreeBuilder` domain service | MODERATE | MEDIUM |
| H6 Direct SQL in Controllers | Enforce repository-only persistence; remove Eloquent from `Http/Controllers` and Mongo from Express routes | HIGH RISK | CRITICAL |
| H8 Domain Boundary Violations | Populate real `Modules/Discovery` & `Modules/Connect`; move Reporting behind published DTOs/ACL | HIGH RISK | CRITICAL |
| H9 Shared Database Coupling | Split Mongo collections per BC; document MariaDB ownership; ban cross-module writes | HIGH RISK | CRITICAL |
| H10 Dual API Duplication | Consolidate on Laravel as single API; proxy or delete Express `dev-api` duplication | HIGH RISK | CRITICAL |
| F5 Legacy Component Patterns | Convert `LegacyMonitorPoller` to hooks; add Error Boundaries; remove unused legacy widget | MODERATE | MEDIUM |

## 1.5 Expected Outcomes

- Controllers become thin HTTP adapters; Application/Domain Services own workflows and are reusable from jobs/CLI.
- Repository interfaces enable testing without MariaDB/Mongo and isolate schema churn to one layer.
- Real bounded contexts (Discovery, Connect, Reporting) stop silent cross-module coupling and support future extraction.
- Eliminating the dual Laravel/Express stack removes the largest source of divergent business rules.
- Frontend domain API modules plus purged legacy components keep the SPA aligned with a single backend contract.

---

## 2. Code Quality & Complexity Analysis

**OVERALL CODEBASE RATING — CODE QUALITY & COMPLEXITY**

# High Risk

Driven by H3 Large Functions (ConnectPage 201 LOC), H5 Duplicate Code (~12.3%), and H9 unsafe extract() (3 call sites).

EXECUTIVE SUMMARY

Klearcom is a small multi-layer platform (Laravel API, React SPA, Node `dev-api`) with no runnable complexity tooling configured (phpstan is listed in Composer but unused; no ESLint complexity rule), so metrics were measured by manual branch/LOC inspection across **39 application source files** (backend 18, frontend 15, `dev-api` 6). The worst findings are a **201-LOC** `ConnectPage` component (ESLint-style CC ≈ 20), **~8.8–12.3%** duplicated business/UI logic (IVR `buildTree` ×3, reachability math ×4, dual Laravel/`dev-api` realtime runners, near-clone Discovery/Connect pages), and **3** unsafe `extract()` call sites in legacy PHP. Git history is short (3 commits, single author `ksabai-gl`); churn and ownership look healthy, but they do not offset the structural duplication and size risks. Overall rating is **High Risk**, driven by large functions, general duplication, and `extract()`.

## 2.1 Benchmark Ratings Summary

Layers covered: **Backend** (Laravel `backend/app` + routes/tests = 18 files) · **Frontend** (React `frontend/src` = 15 files) · **Dev API** (Node `dev-api/src` = 6 files). Tooling: manual LOC/branch counts (no ESLint complexity, no radon/gocyclo, phpstan present in `composer.json` but no config/run).

| # | Hotspot | Primary KPI | GOOD | MODERATE | HIGH RISK | Measured | Rating |
|---|---|---|---|---|---|---|---|
| H1 | High Cyclomatic Complexity | Max complexity per method | <10 | 10–20 | >20 | 20 ( `ConnectPage` , ESLint-style) · FE DiscoveryPage 18 · BE-JS `runConnectTest` 15 · BE-PHP `runConnectTest` 10 | MODERAT |
| H2 | Large Classes | Largest class/file LOC | <300 | 300–1000 | >1000 | 224 ( `dev-api/src/server.js` ); next FE `ConnectPage.tsx` 208 | GOOD |
| H3 | Large Functions | Largest function LOC | <50 | 50–200 | >200 | 201 ( `ConnectPage` ); FE DiscoveryPage 154 · BE max ~64 ( `runConnectTest` ) | HIGH RISK |
| H4 | Business Logic Duplication | Duplicated business logic % | <5% | 5–10% | >10% | ~8.8% ( `buildTree` ×3, reachability×4, dual realtime, KPI×2) | MODERAT |
| H5 | Duplicate Code (general) | Overall duplicate code % | <5% | 5–10% | >10% | ~12.3% (H4 + Discovery/Connect page UI clone) | HIGH RISK |

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| H6 | High Churn Areas | Monthly changes (top files) | <5 | 5–10 | >10 | ~2 (June 2026; top files touched twice) | GOOD |
| H7 | Defect-Prone Files | Fix commits (hottest file) | 1–3 | 4–5 | >5 | 1 ( `ConnectController` / `server.js` / `App.tsx` in fix commits) | GOOD |
| H8 | Ownership Issues | Top-author ownership % | >80% | 60–80% | <60% | 100% ( `ksabai-gl` ) | GOOD |
| H9 | Unsafe `extract()` / dynamic vars (additional) | `extract()` call sites (Good 0 · Moderate 1–2 · High Risk ≥3) | 0 | 1–2 | ≥3 | 3 | HIGH RISK |
| H10 | Missing lifecycle cleanup (additional) | Uncleared interval/SSE components (Good 0 · Moderate 1 · High Risk ≥2) | 0 | 1 | ≥2 | 1 ( `LegacyMonitorPoller` ) | MODERAT |

### Hotspot Score breakdown

| Component | Weight | Sub-score (0–100) | Weighted |
|---|---|---|---|
| Cyclomatic Complexity | 25% | 66 | 16.5 |
| Code Churn | 25% | 18 | 4.5 |
| Defect Density | 20% | 15 | 3.0 |
| Class/Function Size | 15% | 68 | 10.2 |
| Business Logic Duplication | 10% | 72 | 7.2 |
| Developer Ownership Risk | 5% | 8 | 0.4 |
| **Hotspot Score** | **100%** | | **42 / 100** |

## 2.5 Actions Required

| Hotspot | Action | Rating | Priority |
|---|---|---|---|
| H1 High Cyclomatic Complexity | Split `ConnectPage` conditionals into subcomponents; extract shared reachability helpers; enable ESLint `complexity` | MODERATE | MEDIUM |
| H3 Large Functions | Break `ConnectPage` (201 LOC) and shrink `DiscoveryPage` (154 LOC) via hooks + presentational children | HIGH RISK | CRITICAL |
| H4 Business Logic Duplication | Consolidate `buildTree` and reachability math into domain services used by Laravel and `dev-api` | MODERATE | HIGH |

| | | | |
|---|---|---|---|
| H5 Duplicate Code (general) | Extract shared page layout / transcript / query patterns; add duplication detection in CI | HIGH RISK | CRITICAL |
| H9 Unsafe `extract()` | Replace 3 `extract()` sites with explicit validated arrays / DTOs; gate in CI | HIGH RISK | CRITICAL |
| H10 Missing lifecycle cleanup | Add unmount cleanup to `LegacyMonitorPoller` or migrate to React Query | MODERATE | MEDIUM |

## 2.6 Expected Outcomes

- Lower defect rate on Connect/Discovery changes by testing smaller components and a single reachability policy.
- Safer refactors when IVR tree or alert thresholds change — one service update instead of three+ copies.
- Easier code review once page components stay under ~80 LOC and ternaries are flattened.
- Clearer ownership of domain rules ( `IvrTreeBuilder` , `ReachabilityPolicy` ) versus UI shells.
- Elimination of `extract()` and interval-leak debt reduces security and runtime footguns in legacy paths.